

QUDA MultiGrid 的算法理解与 PyQCU 复现

MATPC 紧凑 odd Schur、P/R、Clover/Gauge、33 点粗算子与 setup 优化

代码审计与代数验证

PyQCU · Lattice QCD

2026 年 8 月 30 日

本阶段结论：代数闭环与 MPI 闸门通过，setup 进入批量/显存优化

已完成

QUDA 风格 MATPC：细层一次 odd Schur；粗层直接 RDP。P/R、Clover/Gauge、33 点 stencil、V-cycle、重构与旧实现对照均已落地。

本轮证据

批量局部探测相对逐点为 $0.155\text{ s} \rightarrow 0.122\text{ s}$ ，约 $1.266\times$ ，最大差为 0； $2/4$ rank parity 回环最大差为 0。

资源策略

Schur 窗口矩阵按方向流式切片；工作区不足时当前批次自动减半重试。旧缺陷路径保留，不做无声替换。

完整 D_f

一次 S_o

P_0, R_0

$R_0 S_o P_0$

递归 V-cycle

阶段判断：关键不变量已经可测：奇偶布局保持、 $R = P^\dagger$ 、粗算子保留 local 与 hopping、分布式 33 点 halo 与混合精度运行不破坏真残差；当前主要瓶颈从正确性转为 setup 成本。

来源：实现：pyqcu/solver/_quda_multigrid.py:1094-1292, 1691-2331；优化：pyqcu/tools/_multigrid.py:938-1334；测试：examples/qcu/dev87/。

QUDA 的递归控制流: reset 建层, operator 做一次完整 V-cycle

输入: D_ℓ , B_ℓ , aggregate geometry, precision

reset: 生成/刷新 B_ℓ 与 Transfer_ℓ

$$B_{\ell+1,i} \leftarrow R_\ell B_{\ell,i}$$

createCoarseDirac: $D_{\ell+1} = R_\ell D_\ell P_\ell$

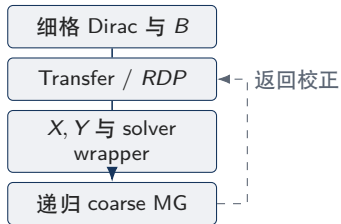
分别保存 residual、smoother 与 sloppy operator

createCoarseSolver: 封装 coarse solver 与递归 MG

operator: prepare $\rightarrow$ pre-smooth $\rightarrow Rr \rightarrow$ recurse

$\rightarrow Pe_c \rightarrow$ post-smooth

输出: solution; 若类型不同则 reconstruct 后重算 residual



控制流含义

粗化只改变表示空间; 每一级仍有完整 operator、平滑器、残差和停止判据。因而“粗层只有 dslash”不等价于 QUDA 的 coarse Dirac。



来源: QUDA: [refer/git-rep/quda/lib/multigrid.cpp](https://github.com/quda/lib/multigrid.cpp):72-197, 1131-1224; PyQCU: [pyqcu/solver/_quda_multigrid.py](https://github.com/pyqcu/solver/_quda_multigrid.py):1915-1986。